# Gestão de Estado

## Android (Kotlin)

Instrutor: Ralf Lima

# 1 - Estados

# O que é

Estado é toda a informação que a UI (*User Interface*) precisa para se mostrar corretamente em determinado momento.

# Tipos

| Tipo de Estado | Onde fica | Persistência |
|---|---|---|
| **State local** | Composable (remember) | Apenas na recomposição |
| **State do ViewModel** | ViewModel (StateFlow) | Persiste em rotação/ recomposição |
| **Evento / Side Effect** | ViewModel (SharedFlow) | Transitório, dispara ação única |
| **Estado derivado** | Composable ou ViewModel | Calculado a partir de outros estados |

# 2 - Class e Data Class

# Class

```
// Classe Pessoa contendo um construtor
class Pessoa(val nome: String, val idade: Int)
```

- Pode ter atributos e métodos;
- Disponibilidade de construtores;
- Suporte a herança / implementação;
- Pode ser uma classe abstrata.

# Data Class

```
// Classe Pessoa contendo um construtor
data class Pessoa(val nome: String, val idade: Int)
```

- Pode ter atributos e métodos;
- Disponibilidade de construtores;
- Suporte a herança / implementação (com restrições);
- Não pode ser uma classe abstrata.

# Data Class - Características

- toString()
  - Classe convencional: com.example.projeto.Classe@b894
  - Data class: Pessoa(nome="Gabriela", idade=34)
- copy()
  - Cria um novo objeto, contendo as características de outro objeto
- equals()
  - É sobrescrito e compara apenas valores e não referência
- hashCode()
  - Data class verifica o valor, classes verificam a referência

# Quando cada um

| | Classe | Data Class |
|---|---|---|
| Finalidade | Criar regras de negócios | Modelos, armazenar dados e DTOs |
| Herança / Interface | Sim | Sim (com restrições) |
| Abstração | Sim | Não |

# 3 - State e MutableStateOf

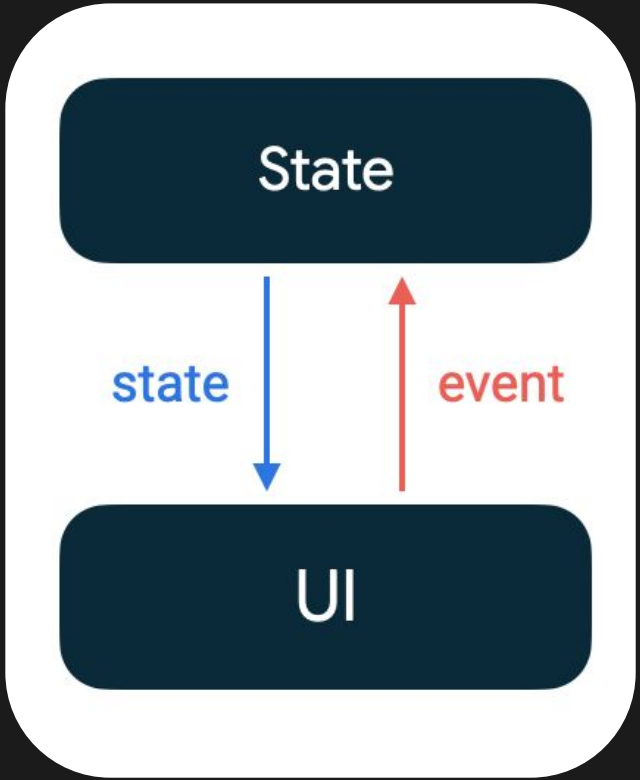
# O que são

**MutableStateOf:** É uma referência de objeto mutável. Utilizamos que o ***Compose*** consiga renderizar dinamicamente as informações.

**State:** É uma referência de objeto apenas para leitura. Utilizamos muito o State quando fazemos uso do **Fluxo de Dados Unidirecionais (UDF)**.

# 4 - Fluxo de Dados Unidirecionais (UDF)

# Funcionamento



# Exemplo prático

```kotlin
data class Mensagem(val texto: String = "Aprendendo Kotlin")
```

# Exemplo prático

```
@Composable
fun MeuComponente() {
    // Cria um estado mutável, inicializado com Mensagem()
    val mensagemMutable = remember { mutableStateOf(Mensagem()) }

    // Expondo o estado como State para a UI
    val mensagemState: State<Mensagem> = mensagemMutable

    // Componente de texto
    Text(
        text = mensagemState.toString(),
        modifier = Modifier.padding(all = 30.dp)
    )
}
```

# 5 - StateFlow

# O que é

**StateFlow** é uma forma reativa de gerenciar estado no Kotlin, feita para você acompanhar mudanças de valor e reagir a elas, de forma parecida com ***LiveData***, mas totalmente baseada em coroutines.

# Exemplo

```
// ViewModel com StateFlow
1 Usage
class CounterViewModel : ViewModel() {
    2 Usages
    private val _count = MutableStateFlow(0)  // Estado interno
    1 Usage
    val count: StateFlow<Int> = _count       // Exposto como StateFlow

    1 Usage
    fun increment() {
        _count.value += 1
    }
}
```

# Exemplo

```
// Composable principal
1 Usage
@Composable
fun CounterScreen(viewModel: CounterViewModel = viewModel()) {
    // Coleta o StateFlow como State do Compose
    val count by viewModel.count.collectAsState()

    // Apenas um componente: Button mostrando o valor
    Button(onClick = { viewModel.increment() }) {
        Text("Contador: $count")
    }
}
```

# Atividade de acompanhamento

Você terá que criar um aplicativo para gerenciar as despesas pessoais. Esse aplicativo deverá ter as seguintes telas:

- Ganhos: Informe todos os ganhos obtidos em determinado mês;
- Gastos: Disponibilize o que foi gasto em determinado mês;
- Sonhos/Desejos: Especifique algo que você queira ter ou fazer, como: viagens, aposentadoria, adquirir um imóvel, etc. Com base no seu valor em conta (ganhos - gastos);
- Página inicial: Exibindo o valor em conta.